

โครงการระบบอัปเดตราคาและสต็อกการ์ดเกมอัตโนมัติ (RPA)

เพื่อเพิ่มประสิทธิภาพการขายแบบ Omnichannel และลดความเสี่ยงจากการผันผวนของราคาตลาด

ชื่อ นายเมธาสิทธิ์ ปัญญาทวีกิจไพศาล 67106154

มหาลัยศรีปทุม

รายงานนี้เป็นส่วนหนึ่งของวิชา

ICT37767

ผู้สอน Jerawan Rickshasuta

บทคัดย่อ (Abstract)

รายงานฉบับนี้จัดทำขึ้นเพื่อนำเสนอการพัฒนาระบบอัตโนมัติด้วยหุ่นยนต์ (Robotic Process Automation: RPA) โดยใช้โปรแกรม UiPath Studio ในการปรับปรุงกระบวนการอัปเดตข้อมูลราคาและคลังสินค้าการ์ดเกมจากระบบฐานข้อมูล Excel เข้าสู่ระบบบริหารจัดการร้านค้าออนไลน์ (Seller Centre) ขององค์กรตัวอย่าง

จากเดิมที่องค์กรใช้พนักงานในการอ่านข้อมูลและคีย์ข้อมูลด้วยมือ (Manual Process) ซึ่งก่อให้เกิดความล่าช้า ไม่ทันต่อความผันผวนของราคาการ์ดเกมในตลาด และเสี่ยงต่อการเกิดข้อผิดพลาดจากมนุษย์ (Human Error) คณะผู้จัดทำจึงได้ออกแบบและพัฒนากระบวนการใหม่ (TO-BE Process) ด้วยการผสานเทคนิค Dynamic Selector และการบริหารจัดการคุณสมบัติ Browser Automation ผลการทดสอบพบว่า ระบบ RPA สามารถทำงานได้ถูกต้องแม่นยำ 100% และลดเวลาการประมวลผลลงได้มากกว่า 95% ช่วยให้องค์กรสามารถบริหารจัดการสต็อกแบบ Omnichannel ได้อย่างมีประสิทธิภาพและตอบสนองต่อราคาตลาดได้อย่างเรียลไทม์

ส่วนที่ 1: ข้อมูลโครงการ (Project Title)

- **ชื่อโครงการ (ภาษาไทย):** ระบบอัปเดตราคาและสต็อกการ์ดเกมอัตโนมัติ (RPA) เพื่อเพิ่มประสิทธิภาพการขายแบบ Omnichannel และลดความเสี่ยงจากการผันผวนของราคาตลาด
- **ชื่อโครงการ (ภาษาอังกฤษ):** Omnichannel Card Game Price & Stock Automation System using RPA
- **แผนก/องค์กรที่ศึกษา:** ร้านค้าการ์ดเกมและของสะสมออนไลน์ยุคใหม่ (Modern Card Retail Platform)

ส่วนที่ 2: ความเป็นมาและวัตถุประสงค์ (Executive Summary)

2.1 กระบวนการทำงานในปัจจุบัน (AS-IS Process)

ในปัจจุบัน พนักงานประจำแผนกคลังสินค้าต้องคอยเปิดไฟล์ Excel ชื่อ [Shop_Inventory.xlsx](#) เพื่อตรวจสอบราคาตลาดล่าสุดและจำนวนการ์ดที่มีอยู่ในคลัง จากนั้นต้องเปิดเว็บเบราว์เซอร์เพื่อล็อกอินเข้าสู่ระบบหลังบ้านร้านค้า และทำการค้นหาการ์ดเกมที่ละรายการ กดปุ่มแก้ไข และพิมพ์ข้อมูลราคาและจำนวนสต็อกลงไปทีละช่องจนครบทุกรายการ

2.2 จุดเจ็บปวดของกระบวนการเดิม (Pain Points)

1. **ความผันผวนของราคาตลาด:** การ์ดเกมเป็นสินค้าที่มีราคาเปลี่ยนแปลงอย่างรวดเร็ว หากอัปเดตราคาบนเว็บไซต์ล่าช้า ร้านค้าจะสูญเสียโอกาสทางธุรกิจ หรือเกิดการขายสินค้าในราคาต่ำกว่าทุน
2. **ปัญหาด้านสต็อกสินค้า (Stockout / Overstocking):** หากสต็อกในหน้าเว็บไม่ตรงกับสินค้าจริง ลูกค้าอาจสั่งซื้อสินค้าที่ไม่มีอยู่จริง ส่งผลต่อคะแนนความพึงพอใจของร้านค้า
3. **ความผิดพลาดจากมนุษย์ (Human Error):** ชื่อการ์ดเกมในภาษาญี่ปุ่นมีความคล้ายคลึงกันมาก ทำให้พนักงานเกิดความล้าและคีย์ข้อมูลสลับช่องหรือกรอกตัวเลขผิดพลาดบ่อยครั้ง

2.3 วัตถุประสงค์โครงการ

- เพื่อใช้เทคโนโลยี RPA (UiPath) ทำงานซ้ำ ๆ แทนพนักงานมนุษย์
- เพื่อเพิ่มความถูกต้องแม่นยำในการซิงค์ข้อมูลคลังสินค้าและราคาให้เป็น 100%
- เพื่อลดระยะเวลาการทำงานจากเดิม 10-15 นาที ให้เหลือต่ำกว่า 30 วินาที

ส่วนที่ 3: Template รายงานสรุป (Mini-PDD)

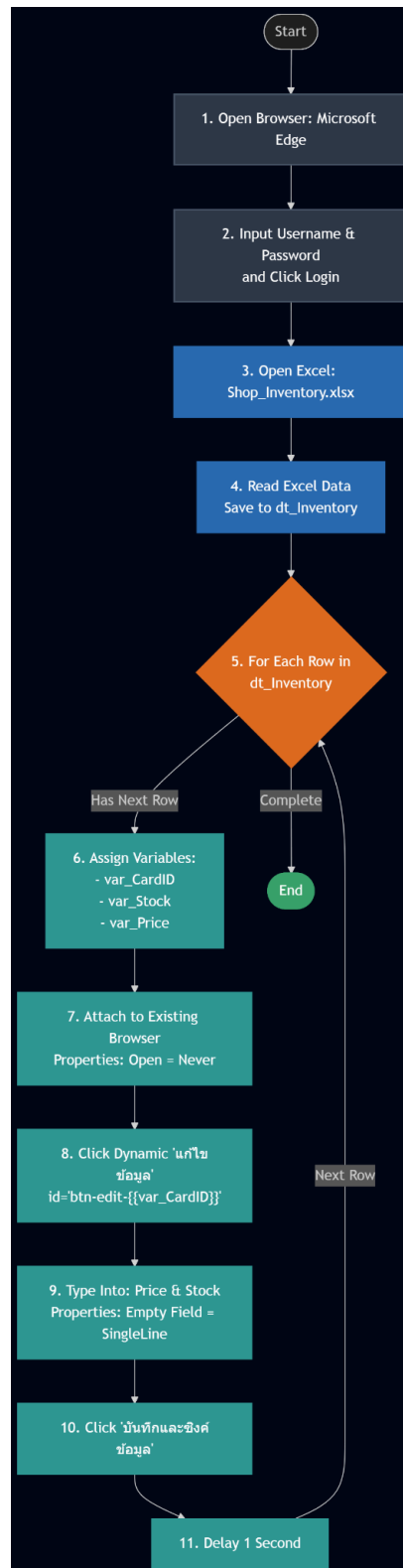
เนื้อหาส่วนนี้อ้างอิงตามโครงสร้างข้อกำหนดรายงานสรุป Mini-PDD ของโครงการ:

1. Process Overview (ภาพรวมกระบวนการ)

- **ชื่อกระบวนการ:** กระบวนการซิงค์ข้อมูลราคาและคลังสินค้าการ์ดเกม (Card Price & Stock Synchronization Process)
- **ความถี่ในการทำงาน:** รายชั่วโมง หรือ ทุกครั้งที่มีการอัปเดตราคากลางในไฟล์ Excel
- **ประเภทกระบวนการ:** รันตามรอบเวลา (Scheduled/Event-Driven Automation)
- **ระบบที่เกี่ยวข้อง:** Microsoft Excel และ Web Application (Seller Centre Platform)

2. Process Map (แผนผังการทำงาน)

- ลำดับขั้นตอนโลจิกการทำงานของบอท UiPath (TO-BE Process) มีดังนี้:



3. Data Requirements (ข้อมูลที่จำเป็นและผลลัพธ์)

- **ข้อมูลนำเข้า (Input Data):** ไฟล์ระบบภายใน **Shop_Inventory.xlsx** โครงสร้างตารางประกอบด้วยคอลัมน์:
 - **Card_ID:** รหัสเฉพาะของการ์ด (เช่น GD04-066)
 - **Card_Name_JP:** ชื่อการ์ดภาษาญี่ปุ่น
 - **Total_Stock:** จำนวนสินค้าคงคลังปัจจุบัน
 - **Price/1 (Baht):** ราคาขายล่าสุดต่อหน่วย
- **ผลลัพธ์ของกระบวนการ (Output Data):**
 - ข้อมูลบนระบบหน้าเว็บ Seller Centre ถูกปรับปรุงค่าตรงตาม Excel
 - แสดงแถบแจ้งเตือน Toast Message สีเขียวบนระบบหน้าเว็บเพื่อยืนยันความสำเร็จ

4. Technical Specification (รายการ Activities หลัก)

ระบบนี้พัฒนาขึ้นโดยใช้แพ็คเกจ UiPath Modern Activity ทั้งสิ้น เพื่อความเสถียรและความรวดเร็ว ประกอบด้วย:

1. **Use Application/Browser:** ใช้สำหรับเปิดเบราว์เซอร์ Edge เพื่อเข้าหน้าเว็บ และใช้ซ้ำภายในลูปโดยตั้งค่า **Open = Never** เพื่อเกาะหน้าตาเดิมทำงานต่อโดยไม่แดงกลับหน้าล็อกอิน
2. **Use Excel File & Read Range:** ทำหน้าที่ดึงข้อมูลจากไฟล์ Excel แปลงเป็นข้อมูลประเภท Data Table เก็บในตัวแปร **dt_Inventory**
3. **For Each Row in Data Table:** ควบคุมการวนลูปเพื่ออ่านข้อความและส่งคำสั่งข้อมูลทีละแถวอัตโนมัติ
4. **Assign:** ใช้สำหรับแมปข้อมูลจากคอลัมน์ในตาราง ออกมาเป็นตัวแปรประเภทข้อความ (String) ได้แก่ **var_CardID**, **var_Stock**, และ **var_Price**
5. **Click (Dynamic Selector):** สั่งการคลิกปุ่มแก้ไขตัวสินค้า โดยมีการปรับแก้โค้ด Strict Selector ให้นำตัวแปรมาใช้งานในรูปแบบ **id='btn-edit-{{var_CardID}}'** และนำคุณสมบัติล็อกแถวอย่าง **tableRow** ออก เพื่อให้บอทค้นหาปุ่มกดได้อย่างยืดหยุ่น
6. **Type Into:** ใช้ป้อนข้อมูลสต็อกและราคา โดยเปิดคุณสมบัติ **Empty Field = SingleLine** เพื่อล้างข้อมูลเก่าทิ้งก่อนเริ่มพิมพ์ทุกครั้ง

5. Test Result (ผลการทดสอบระบบ)

จากการรันทดสอบระบบในสภาพแวดล้อมจำลอง (Test Environment) สรุปผลได้ดังนี้:

- **จำนวนรายการทดสอบทั้งหมด (Total Cases):** 8 รายการสินค้า (ตามฐานข้อมูล Excel)
- **จำนวนรายการที่สำเร็จ (Passed Cases):** 8 รายการ
- **จำนวนรายการที่ล้มเหลว (Failed Cases):** 0 รายการ
- **อัตราความสำเร็จ (Success Rate):** 100%

- **ความเร็วเฉลี่ยรวม:** ต่ำกว่า 30 วินาที ตลอดทั้งกระบวนการ (เฉลี่ยประมาณ 2-3 วินาทีต่อการรัน 1 ใบ ซึ่งรวมเวลา Delay หน้าจอแล้ว)

ส่วนที่ 4: บทเรียนที่ได้รับและการพัฒนาต่อยอด (Lessons Learned)

4.1 อุปสรรคที่พบระหว่างการพัฒนาและวิธีแก้ไข

- **ปัญหาหน้าเว็บรีเฟรชกลับไปหน้าแรก:** ในช่วงแรกพบปัญหาเว็บจำลองที่ใช้ URL เดียวกันทั้งระบบ ส่งผลให้เมื่อจบการทำงานในลูปลแล้วระบบพยายามเปิดแท็บใหม่หรือรีเฟรชหน้าจอ ทำให้เซสชันหลุดและต้องกลับไปหน้าล็อกอินเสมอ
 - วิธีแก้ไข: ทำการปรับคุณสมบัติของบล็อก **Use Application/Browser** ใน Properties กลุ่ม Options โดยเปลี่ยนค่าของ **Open ให้เป็น Never** เพื่อสั่งให้หุ่นยนต์ทำงานต่อเนื่องบนหน้าต่างเดิมที่เปิดค้างอยู่ทันที
- **ปัญหา Selector ล็อกตำแหน่งแถว:** บอทดคลิกแก้ไขเฉพาะสินค้าตัวเดิมไม่ยอมเปลี่ยนแถว
 - วิธีแก้ไข: เข้าไปแก้ไขโครงสร้างโค้ด XML ใน Strict Selector โดยตัดคำสั่ง **tableRow='2'** ทิ้ง แล้วนำตัวแปร **var_CardID** ไปฝังแบบ Dynamic แทน ส่งผลให้บอทมีความยืดหยุ่นสูงขึ้น

4.2 แนวทางการต่อยอดระบบในอนาคต (Future Scope)

ในอนาคต สามารถเพิ่มประสิทธิภาพโครงการได้โดยการเพิ่ม Activity **HTTP Request** หรือสร้างบอทอีกหนึ่งตัวให้เข้าไปทำหน้าที่สแกน (Web Scraping) ราคาของการ์ดเกมจากเว็บไซต์ซื้อขายระดับสากล (เช่น Yuyu-Tei หรือ TCGPlayer) เพื่อดึงราคาล่าสุดมาคำนวณและอัปเดตลงในไฟล์ Excel **Shop_Inventory** โดยอัตโนมัติก่อน[cite: 1] จากนั้นจึงส่งต่อไปบอทตัวนี้ทำหน้าที่ซิงค์ข้อมูลขึ้นระบบหลังบ้าน ทำให้ระบบสามารถปรับเปลี่ยนราคาตามกลไกตลาดโลกได้แบบอัตโนมัติอย่างสมบูรณ์ (Fully Automated End-to-End Process)